BusBidder Spain: Filtering and Ranking Competing Bus Offers for Individual Travellers

Final project report, ACIT4420 Problem-solving with scripting (2026)

Mehmood Hasan

Student number: add your student number

Department of Computer Science, OsloMet – Oslo Metropolitan University

Source code: add GitHub repository link and final commit hash

Date: submission date

Abstract

Travellers who compare bus journeys rarely look for a single property. The cheapest ticket may involve a transfer, a long journey and a fare that cannot be changed, while a slightly more expensive ticket may suit a particular traveller much better. This report describes the design, implementation and evaluation of BusBidder Spain, a modular Python application that collects simulated offers from five fictional bus companies serving fifteen destinations from Madrid, removes the offers that break a traveller's mandatory requirements and ranks the remaining offers with a transparent weighted scoring model. The application validates 225 offers loaded from files, places invalid records aside in a quarantine file, normalises each criterion to a 0–100 scale, explains the top recommendation and simulates a booking whose seat reductions persist between sessions. The model was tested with 38 automated tests and evaluated with the specification's own example request and four traveller profiles. The profiles produced clearly different and largely reasonable recommendations. The evaluation also showed that the model is sensitive in ways that are not visible to the user: the example request was decided by 0.03 points, adding one non-winning offer to the candidate pool changed the winner, and one company never won any of 75 searches. I therefore argue that the recommendation is defensible only as the best compromise under the chosen constraints, weights and normalisation method, and I discuss how the model could be made fairer and more stable.

Keywords: multi-criteria decision making; weighted sum model; normalisation; explainable recommendation; Python; data validation

Contents

1 Introduction 4
1.1 Background 4
1.2 Central question and objectives 4
1.3 Structure of the report 4
2 Problem interpretation and requirements 4
2.1 Interpreting the task 4
2.2 Requirements and where they are met 5
2.3 Assumptions and scope 5
3 Software design and architecture 6
3.1 Overall architecture 6
3.2 Module structure 6
3.3 Classes and how they collaborate 7
3.4 Inheritance and composition 7
3.5 Metaprogramming: descriptors and a criteria registry 8
3.6 Regular expressions 8
4 Dataset and validation 8
4.1 Design of the simulated dataset 8
4.2 Validation rules 9
4.3 Rejected, corrected or placed aside? 9
4.4 Error categories and logging 10
5 Filtering and ranking model 10
5.1 Hard constraints and soft preferences 10
5.2 Normalising the criteria 11
5.3 Weights 12
5.4 Ties, missing values and extreme values 12
5.5 Preventing one criterion from dominating 12
5.6 Generating the explanation 12
6 Simulated booking 13
7 Testing and experimental evidence 13
7.1 Automated tests 13
7.2 The specification's example request 13
7.3 Comparison of four traveller profiles 14
7.4 Are the rankings reasonable? 15
7.5 Robustness checks 16

8 Evaluation, limitations and possible unfair outcomes 16
8.1 Min–max normalisation amplifies small differences 17
8.2 Rankings depend on the candidate pool 17
8.3 Hard constraints create sharp edges 17
8.4 Partial credit for flexibility can be lost 17
8.5 Possible unfair outcomes 17
8.6 Limitations of the data and the implementation 18
8.7 What I would change 18
9 Reproducibility 18
10 Declaration of generative AI use 19
11 Conclusion 19
References 21
Appendix A. Evidence files 21

1 Introduction

1.1 Background

Planning a bus journey often means comparing several companies, departure times, prices and ticket conditions at the same time. Sorting by a single attribute, such as the lowest price, is easy to implement but hides the trade-offs that matter to travellers. A low fare may come with a transfer, a departure at an inconvenient hour or a ticket that cannot be changed, and a traveller on a business trip will value these properties very differently from a backpacker on a tight budget. Comparing offers on several criteria at once is a classic multi-criteria decision problem, for which the weighted sum model is one of the simplest and best-established methods (Hwang & Yoon, 1981; Triantaphyllou, 2000). Because the model adds the criterion values together, the criteria must first be brought to a common scale, which this project does by normalisation (Section 5.2).

BusBidder Spain is a fictional marketplace in which a traveller describes a flexible travel request and five simulated bus companies return offers. The task of the project was to design and implement a Python application that first removes offers that violate the traveller's mandatory conditions and then ranks the remaining offers with a transparent weighted scoring model. The specification stresses that the program must help the traveller understand not only which offer ranks highest, but also why it ranks highest and which disadvantages remain.

1.2 Central question and objectives

The project addresses the central question given in the specification: *how can a Python application filter and rank competing bus offers according to the requirements and preferences of an individual traveller?* To answer the question, I set four objectives. The first was to build a modular, object-oriented application that loads and validates the marketplace data from files. The second was to separate hard constraints from soft preferences in a way that is visible to the user. The third was to implement a reproducible weighted ranking with an explanation generated from the data. The fourth was to evaluate whether the resulting rankings are reasonable for different travellers, and where they are not.

1.3 Structure of the report

Section 2 interprets the task and maps each requirement to the implementation. Section 3 describes and justifies the software design, and Section 4 describes the dataset and its validation. Section 5 explains the filtering and ranking model, including the design decisions required by the specification, and Section 6 describes the simulated booking. Section 7 presents the tests and the experimental results, and Section 8 evaluates the model critically, including its limitations and possible unfair outcomes. Section 9 explains how the results can be reproduced, Section 10 declares the use of generative AI, and Section 11 concludes.

2 Problem interpretation and requirements

2.1 Interpreting the task

I interpreted the task as a two-stage decision pipeline. In the first stage, every offer is checked against conditions that the traveller cannot accept being broken, such as the destination, the travel window, the number of free seats or a maximum price. An offer that fails any of these conditions is

removed and never scored. In the second stage, the remaining offers are compared on criteria where the traveller is willing to trade one property against another, and the comparison is expressed as a weighted score. This separation matters because a weighted score can always be compensated: a very cheap ticket can make up for a poor departure time. Conditions that must never be compensated therefore cannot be part of the score.

I also read the specification as requiring honesty about the result. The top-ranked offer is the best offer only under the selected constraints, weights and scoring assumptions, and the program should say so. For this reason, the evaluation in Sections 7 and 8 focuses as much on when the ranking is fragile as on whether it produces sensible winners.

2.2 Requirements and where they are met

Table 1 lists the requirements of the specification, where each one is implemented and where the evidence can be found. The evidence files are stored in the `outputs/` folder of the repository and are listed in Appendix A.

Table 1. Requirements of the specification and where they are met.

Requirement (specification section)	Implementation	Evidence
Object-oriented model in several modules (Task 1, §7)	Eight packages under `src/` with model, loader, engine, booking and interface classes	Section 3, Figure 1
Diagram of components and information flow (Task 1, §9)	`report/architecture_diagram.png`	Figure 1
Load and validate data from files (Task 2)	`DataLoader`, descriptors, regex checks, quarantine CSV	Section 4, `05_quarantine_review.txt`
Custom exceptions and error categories (Task 2, §7)	`BusBidderError` hierarchy, including `DataFileMissingError`	Section 4.4, `07_missing_file_error.txt`
Log file of important events (Task 2)	`busbidder.log` written by `src/loaders/logger.py`	`08_busbidder_log_excerpt.log`
Hard constraints separated from preferences, visible to the user (Task 3)	`FilterEngine`; hard/soft summary printed before every ranking	Section 5.1, Table 6
Weighted ranking with at least four criteria (§5)	Six registered criteria; every profile uses four	Section 5.2
Normalisation, direction, weight validation, ties, missing and extreme values, dominance (§5)	`normalizers.py`, `TravellerRequest.validate_weights`, `RankingEngine`	Sections 5.2–5.5
Ranked list with values, criterion scores, total score and explanation (§6)	Two tables and an explanation panel per search	`01_demo_four_profiles.txt`
At least three traveller profiles (§6)	Five profiles, four used in the comparison	Section 7.3
Simulated booking with reference, capacity and fare conditions (§6)	`BookingManager`, `BookingRecord`, `data/bookings.json`	Section 6, `03_interactive_booking.txt`
Metaprogramming technique (§7)	Validation descriptors and the `@register_criterion` registry	Section 3.5
Tests of normal, invalid, missing-data and boundary cases (§7)	38 pytest tests in `tests/`	Section 7.1, `06_test_results.txt`

2.3 Assumptions and scope

All journeys are one-way and depart from Madrid. The dataset covers the travel window from 12 to 14 July 2026, which matches the example request in the specification. Offers are only ever compared with other offers to the same destination, because a search is always for one destination. Bookings are simulated: no payment information is collected, and a confirmed booking simply reduces the number of free seats. The optional booking extensions listed in the specification, such as

waiting lists or a graphical interface, were not implemented; the project concentrated on the core requirements and on the evaluation of the ranking model.

3 Software design and architecture

3.1 Overall architecture

The application is organised as a pipeline in which each stage has one responsibility and passes plain objects to the next stage (Figure 1). Data files are read and validated by the loader, which returns valid `Offer` objects and sends invalid rows to a quarantine file. The traveller's request is represented by a `TravellerRequest` object that holds both the hard constraints and the soft preferences. The filter engine removes unacceptable offers and records why they were removed, the ranking engine scores the remaining offers, the explainer describes the winner, and the booking manager turns a chosen offer into a stored booking. The command-line interface only coordinates these objects and prints their results.

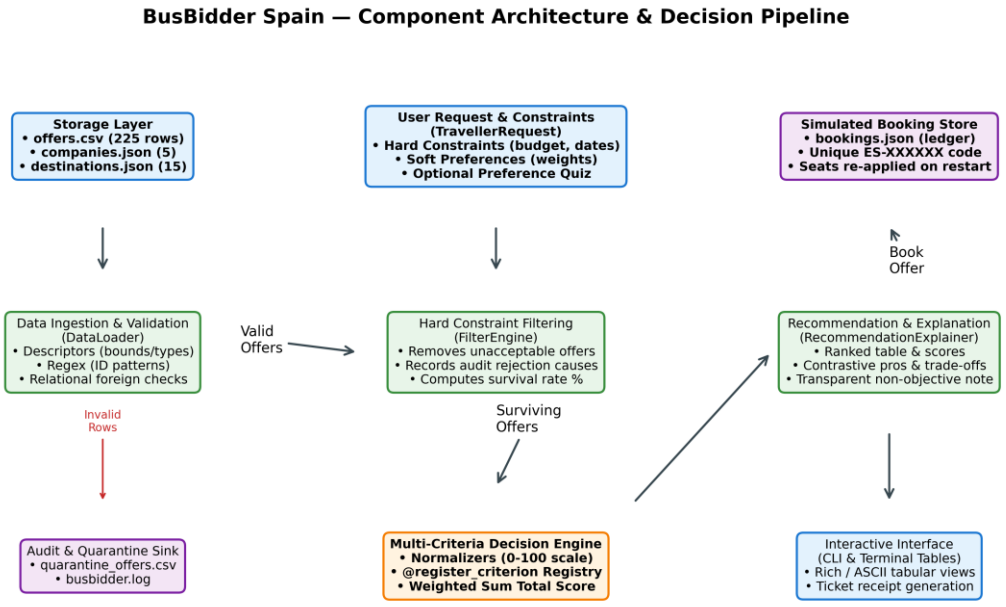


Figure 1. Main components of BusBidder Spain and the flow of information between them.

I chose a pipeline because it mirrors the decision process described in Section 2.1 and because it makes each stage testable in isolation. The filter engine, for example, can be tested with a handful of hand-made offers without loading any files, and the ranking engine never needs to know where the offers came from.

3.2 Module structure

Table 2 summarises the packages and their responsibilities. The entry point, `main.py`, contains the command-line options and the interactive dialogue, while all domain logic lives in the `src/` package.

Table 2. Packages and their responsibilities.

Package	Main classes and functions	Responsibility
`src/models`	`Offer`, `Company`, `Destination`, `TravellerRequest`, `BookingRecord`	Domain objects and their own consistency rules
`src/validators`	Exception hierarchy, regex patterns, descriptors	Reusable validation and error categories
`src/loaders`	`DataLoader`, logger, dataset generator	Reading files, quarantine and logging
`src/engine`	`FilterEngine`, normalisers, criteria registry, `RankingEngine`, `RecommendationExplainer`	Decision logic
`src/booking`	`BookingManager`	Booking references, capacity and the booking ledger
`src/profiles`	Five profile factory functions	Predefined traveller preferences
`src/extensions`	Preference questionnaire	Turning 1–5 ratings into weights
`src/cli`	Tables, hard/soft summary, demonstration	Presentation only

3.3 Classes and how they collaborate

The central class is `Offer`, which represents one departure from one company on one date. It validates itself when it is created, so an `Offer` object that exists is always internally consistent. `Company` and `Destination` hold the catalogue data that the loader uses to check that every offer refers to a known company and a known destination. `TravellerRequest` represents the traveller and the search; it keeps hard constraints and soft preferences in clearly separated attributes and validates the weights and the travel window when it is created.

The engine classes are deliberately stateless. `FilterEngine` returns a `FilterResult` that contains the accepted offers and a count of rejections per constraint. `RankingEngine` returns a list of `RankedOffer` objects, each of which holds the original offer together with its criterion scores, total score and rank. `RecommendationExplainer` reads this list and produces the explanation. `BookingManager` owns the collection of `BookingRecord` objects and is the only class that writes the booking file. Keeping file access in two classes, the loader and the booking manager, means that the rest of the application can be tested without touching the disk.

3.4 Inheritance and composition

I used inheritance in two places where the subclasses really are specialised versions of their parent. The first is the exception hierarchy. Every application error inherits from `BusBidderError`, with separate branches for data validation, ranking configuration and booking errors. This allows `main.py` to catch all known application errors in one place and print a short message instead of a traceback, while tests can still check for the exact error class. `DataFileMissingError` inherits from both `BusBidderError` and Python's `FileNotFoundError`, so that code expecting the built-in error continues to work. The second place is the validation descriptors, which share a common base class that stores the attribute name.

Elsewhere I preferred composition, following the common advice to favour object composition over class inheritance (Gamma et al., 1995). A `RankedOffer` contains an `Offer` rather than inheriting from it, because a ranking is a view of an offer in the context of one search, not a different kind of offer. The same offer appears with different scores and ranks for different travellers, which would be awkward if the score were part of the offer itself. Similarly, a `BookingRecord` copies the fare conditions of the offer at the moment of booking, so that a later change in the catalogue does not silently change the conditions of a ticket that has already been sold.

3.5 Metaprogramming: descriptors and a criteria registry

The specification asks for at least one metaprogramming technique. I used two, each because it removed a concrete problem rather than to satisfy the requirement. The first is a set of data descriptors (`PositiveNumber`, `NonNegativeInt`, `BoundedFloat` and `RegexValidatedString`). A descriptor is an object that customises attribute lookup, storage and deletion, usually by defining `__get__` and `__set__` (Python Software Foundation, n.d.; Ramalho, 2022). Declaring `price = PositiveNumber()` on the `Offer` class means that every assignment to `price`, whether in the constructor, in a test or later in the program, is checked by the same code. Without descriptors, the same checks would have to be repeated in property setters or in the constructor, and an assignment made after construction, such as a seat reduction, would bypass them.

The second technique is a registry built with a decorator. Each scoring criterion is an ordinary function decorated with `@register_criterion("name")`, which stores the function in a dictionary. The ranking engine looks up the criteria named in the traveller's weights and never refers to a specific criterion. A new criterion can therefore be added by writing one decorated function, without changing the ranking engine, which is an application of the open–closed principle (Meyer, 1988). The registry also allows the engine to reject a weight that refers to an unknown criterion with a clear `RankingConfigurationError`, instead of failing later with a `KeyError`.

3.6 Regular expressions

Regular expressions are used only for structured identifiers and formats, where they are the natural tool. Offer identifiers must match `^OFF-[A-Z]{2,4}-\d{4}$`, booking references must match `^ES-[A-Z0-9]{6}$`, times must be valid 24-hour `HH:MM` values and dates must follow the ISO format `YYYY-MM-DD`. The same patterns validate the data files and the traveller's input in the interactive mode. I did not use regular expressions for numeric ranges or for business rules such as "arrival after departure", because those are clearer and safer to express as ordinary comparisons.

4 Dataset and validation

4.1 Design of the simulated dataset

The marketplace consists of three files in the `data/` folder. `companies.json` describes the five companies, `destinations.json` describes the fifteen destinations with their road distance from Madrid, region and whether they are on the coast, and `offers.csv` contains the offers. The offers were produced by a deterministic generator (`src/loaders/generate_datasets.py`), so the dataset can be recreated exactly. Each company serves every destination on each of the three days, which gives 5 × 15 = 75 company–destination combinations, as required by the specification, and 225 offers in total.

The suggested company profiles were turned into concrete and different fare rules, summarised in Table 3. The price of an offer is the distance multiplied by a company-specific rate, with a €5 surcharge for journeys with a transfer and a minimum fare of €14. The duration is based on an average speed of 78 km/h, multiplied by a company-specific factor, and the emissions per passenger are proportional to the distance. The number of free seats varies between 15 and 49 per offer.

Table 3. Fare rules used to generate the offers of each company.

Company	Departures	Transfers	Fare conditions	Luggage	Rating	Price per km	CO_2 per km
SolRuta	07:45, 11:30, 15:15	1 if > 450 km	Neither changeable nor refundable	12 kg	3.9	€0.055 (+€5 with transfer)	0.038 kg
Camino Express	08:30, 14:00, 17:30	Always direct	Changeable and refundable	20 kg	4.7	€0.095	0.032 kg
CostaLink	09:15, 13:45, 16:30	Direct to the coast; 1 inland if > 400 km	Changeable only	25 kg	4.3	€0.068 coast, €0.082 inland	0.035 kg
VerdeBus	10:00, 15:00	Always direct	Changeable and refundable	18 kg	4.8	€0.075	0.014 kg
NocheVia	22:30, 23:45	Always direct	Changeable only	20 kg	4.2	€0.065	0.030 kg

The rules were chosen so that no company dominates on every criterion. SolRuta is the cheapest on most routes but the slowest and least flexible, Camino Express is the fastest and most flexible but the most expensive, VerdeBus has by far the lowest emissions and the best rating, CostaLink offers the most luggage, and NocheVia travels overnight. Without such trade-offs, every traveller would receive the same recommendation and the ranking model could not be evaluated.

4.2 Validation rules

Every row of the offer file is converted to the correct types and checked before it becomes an `Offer` object. Table 4 lists the checks. Most of them are carried out by the descriptors and the regular expressions described in Section 3; the checks of company and destination are carried out by the loader against the catalogue files. To verify that the checks work on real files, I prepared a separate file, `data/corrupt_offers_sample.csv`, with five deliberately broken rows, one for each of the main error types named in the specification.

Table 4. Validation checks applied to each offer.

Check	Mechanism	Example rejected in the corrupt sample
Offer identifier has the expected structure	Regular expression	`INVALID-ID`
Company and destination exist in the catalogue	Lookup in the loaded JSON files	Destination "Atlantis"
Date and times have valid formats	Regular expression and calendar parsing	–
Price and duration are positive numbers	`PositiveNumber` descriptor	Price of €−25.50
Transfers and free seats are whole numbers ≥ 0	`NonNegativeInt` descriptor	–
Customer rating lies between 1.0 and 5.0	`BoundedFloat` descriptor	Rating of 7.5
Departure, arrival and duration agree (overnight journeys allowed, 30 minutes tolerance)	Check in the `Offer` constructor	23:00 to 05:00 stated as 120 minutes
Required values are present	Type conversion fails on empty fields	Tested with empty price and departure time

4.3 Rejected, corrected or placed aside?

The specification asks whether invalid records are rejected, corrected or placed aside for review. I chose to place them aside. An invalid row is never loaded, but it is written to `data/quarantine_offers.csv` together with its line number and the reason for the rejection, and the

event is logged. The remaining rows continue to load, so one broken record does not stop the marketplace. I decided against automatic correction because, for this kind of data, a correction is a guess. A negative price could be a sign error or a refund, and a duration that does not match the timetable could mean that either value is wrong. Showing a traveller an offer based on a guessed price would be worse than not showing the offer at all, and a human reviewer can repair the row and reload the file.

There are two deliberate exceptions. First, an offer with zero free seats is treated as valid data, because a sold-out departure is a normal state of a timetable and not an error in the file. Such an offer is removed by the seat constraint during filtering instead, which a separate test confirms. Second, if the emissions column is missing from the file altogether, the loader uses a default of 12 kg per passenger. This is the only correction the loader makes, and it is a weakness, since a missing environmental measure is then treated as an average one; Section 8 returns to this point. During the final review it was also found that the free-text cancellation policies in `companies.json` contradicted the offer data (for example, SolRuta was described as allowing changes for a fee). These texts are not used by the program and are therefore not validated, but they were corrected so that the files describe the same marketplace.

4.4 Error categories and logging

Errors are grouped into three categories that correspond to the stages of the pipeline: data errors (`DataValidationError` and `DataFileMissingError`), configuration errors in the request (`ConstraintViolationError`, `WeightValidationError`) and booking errors (`SeatUnavailableError`, `InvalidBookingReferenceError`). Data errors on single rows are handled by quarantine, while a missing data file stops the program with a short message and exit code 1, since the marketplace cannot operate without it. Every important event is recorded with a timestamp in `busbidder.log`, including the number of offers loaded and quarantined, the outcome of each filtering and ranking step, and every booking and cancellation.

5 Filtering and ranking model

5.1 Hard constraints and soft preferences

Table 5 shows how I divided the traveller's conditions. My rule was that a condition is a hard constraint when breaking it makes the offer useless or unacceptable to the traveller regardless of its other qualities, and a soft preference when a better value on another criterion could reasonably make up for it. The destination, the travel window and the free seats are always hard. The remaining hard constraints are optional, so that the traveller decides whether, for example, a budget is a firm limit or merely a wish.

Table 5. Division of the traveller's conditions into hard constraints and soft preferences.

Condition	Type	Justification
Destination and travel window	Hard (always)	An offer to another city or on another day does not answer the request.
Enough free seats	Hard (always)	An offer that cannot be booked is not a real option.
Maximum price	Hard (optional)	A budget limit is usually a firm limit; below the limit, price is also a soft criterion.
Maximum transfers, maximum duration, minimum luggage	Hard (optional)	These are practical limits, for example a traveller with heavy luggage cannot use a 12 kg allowance.
Changeable or refundable ticket required	Hard (optional)	When required, a non-flexible fare is unacceptable; otherwise flexibility is scored.
Price, duration, departure time, flexibility, rating, emissions	Soft (weighted)	Better values are preferred but can be traded against each other.

Two conditions appear on both sides, and this is intentional. A budget limit removes offers above the limit, but among the offers that remain, a cheaper offer is still preferred. In the same way, a traveller who requires a changeable ticket removes non-changeable fares, but may still prefer a fare that is also refundable. To make the distinction visible, the program prints a summary before every ranking in which the hard constraints and the soft preferences are listed under separate headings, and it reports how many offers each constraint removed. The interactive mode asks for the hard constraints and the soft preferences in separate steps with the same headings.

5.2 Normalising the criteria

The criteria are measured in different units, so their raw values cannot be added. Each criterion is therefore converted to a score between 0 and 100 before weighting. For criteria where a higher value is better, I used min–max normalisation over the offers that survived the filter. For criteria where a lower value is better, the same formula is reversed, so that the best value always receives 100:

$$\text{lower is better: } \text{score} = (\max - x) \,/\, (\max - \min) \times 100$$

$$\text{higher is better: } \text{score} = (x - \min) \,/\, (\max - \min) \times 100$$

Departure-time convenience is handled differently, because it has no natural minimum or maximum. It is based on the distance between the departure and the preferred time, measured around a 24-hour clock so that 23:00 and 01:00 are two hours apart, not twenty-two. The score falls linearly from 100 at the preferred time to 0 at eight hours or more:

$$\text{departure score} = \max(0, (8 - d) \,/\, 8 \times 100), \text{ where } d = \min(|h - t|, 24 - |h - t|)$$

Flexibility is categorical. An offer receives 0.4 points if it is changeable and 0.6 points if it is refundable, so a fully flexible fare scores 1.0, and these values are then normalised like any other higher-is-better criterion. Table 6 summarises the six criteria that are registered in the application. Every profile uses four of them, and the total score is the weighted sum of the four criterion scores:

$$\text{Score}(j) = \Sigma\, w_i \cdot S_i(j), \text{ with } \Sigma\, w_i = 1$$

Table 6. Criteria available for ranking.

Criterion	Raw value	Direction	Normalisation
Price	Euro	Lower is better	Reversed min–max
Duration	Minutes	Lower is better	Reversed min–max
Departure time	Hours from preferred time	Closer is better	Fixed 8-hour scale on a 24-hour clock
Flexibility	0, 0.4, 0.6 or 1.0	Higher is better	Min–max
Customer rating	1.0–5.0 stars	Higher is better	Min–max
Emissions	kg CO_2 per passenger	Lower is better	Reversed min–max

5.3 Weights

Weights are stored as fractions in a dictionary. When a `TravellerRequest` is created, the weights are checked: none may be negative and together they must sum to 1.0, with a tolerance of 0.001 for floating-point rounding. A request that fails these checks raises a `WeightValidationError` and is never ranked. Travellers can obtain weights in two ways. They can choose one of the predefined profiles, or they can answer a short questionnaire that rates five factors from 1 (not important) to 5 (crucial). The questionnaire divides each rating by the sum of all ratings, which always gives valid weights, and shows the resulting percentages before the ranking.

5.4 Ties, missing values and extreme values

Ties. Offers with equal total scores, compared to four decimal places, are ordered by lower price, then higher rating, then shorter duration. If these are also equal, Python's stable sort keeps the order of the catalogue, so the earlier departure in the file is listed first. The ordering is therefore fully reproducible. When the top two offers are tied, the explanation says so and names the rule that decided the order, as shown in Section 7.3. **Missing values** are prevented from reaching the ranking by the validation in Section 4: an offer with a missing price or time is placed aside, so every scored offer has a value for every criterion. If all remaining offers have the same value for a criterion, the range is zero; in that case all offers receive 100 for that criterion, which avoids division by zero and does not change the order. **Extreme values** are bounded by validation, since impossible values such as negative prices or ratings above five are rejected. Values that are possible but unusual, such as a very expensive offer, are not removed, because they may be exactly what a business traveller wants. Their effect on the other scores is discussed in Section 8.

5.5 Preventing one criterion from dominating

Three measures limit how much influence a single criterion can have. Normalisation puts every criterion on the same 0–100 scale, so that price in euros cannot overwhelm a rating measured in stars simply because its numbers are larger. The weights must sum to one, so a criterion's maximum contribution equals its weight. Finally, the departure score uses a fixed scale rather than the range of the candidate pool. I want to be clear, however, that these measures do not prevent unintended dominance completely. Min–max normalisation stretches any difference, however small, to the full 0–100 range, so a criterion on which the offers barely differ can still decide the ranking. Section 8.1 shows that this happened in the experiments.

5.6 Generating the explanation

The explanation of the winning offer is generated from the data rather than from fixed text. The explainer compares the winner with all remaining offers and lists its advantages, such as the lowest price, the fastest journey, a direct route or a fully flexible fare, together with its remaining disadvantages, such as how many euros more it costs than the cheapest offer or how many minutes longer it takes than the fastest one. Miller (2019) shows that people ask why one outcome occurred instead of another, and that they expect an explanation to name one or two main causes rather than a complete account. Following these two findings, the explanation then compares the winner with the second-ranked offer, criterion by criterion, and names at most the two criteria that gave it the largest lead in weighted points. If the lead is smaller than two points, the explanation warns that a small change in the weights could reverse the order. Every explanation ends with a note that the offer is not objectively superior, but the best fit under the traveller's constraints, scoring rules and weights.

6 Simulated booking

After viewing the ranking, the traveller can select an offer by its rank, enter a name and choose the number of seats. The booking manager first checks that enough seats remain and raises a `SeatUnavailableError` if they do not. It then reduces the number of free seats, generates a booking reference that matches `^ES-[A-Z0-9]{6}$` and is not already in use, stores the fare conditions of the offer (changeability, refundability and luggage allowance) in the booking, and writes the booking to `data/bookings.json`. A formatted ticket is printed as confirmation. No payment information is requested.

Keeping capacity consistent required one design decision. The offer file is the original catalogue, and I did not want the program to rewrite it after every booking. Instead, the booking file acts as a ledger. Each time the program starts, the seats in all confirmed bookings are subtracted from the freshly loaded offers, so the capacity shown to the next traveller is always the catalogue capacity minus the seats already sold. A cancelled booking is no longer subtracted, which returns its seats. In the evidence run, booking two seats on offer OFF-SR-1001 reduced its capacity from 15 to 13, and the reduction was still present after a restart (Appendix A, files 03 and 04).

7 Testing and experimental evidence

7.1 Automated tests

The application is covered by 38 automated tests written with pytest, all of which pass (Table 7). The tests cover the four kinds of case named in the specification. Normal cases include loading the complete catalogue, filtering by each constraint, ranking, and a full booking. Invalid cases include negative prices, malformed identifiers, ratings outside the scale, inconsistent timetables, weights that do not sum to one and travel windows that end before they start. Missing-data cases include a missing data file and rows with an empty price or departure time. Boundary cases include a criterion on which all offers are equal, departures on either side of midnight, an offer with zero seats, booking more seats than remain, and identical questionnaire ratings.

Table 7. Automated tests per module.

Test module	Tests	What is tested
`test_descriptors_and_models.py`	8	Valid offers; negative price, bad identifier, rating out of range and timetable errors; invalid and negative weights; reversed travel window
`test_loaders.py`	6	Catalogue files, valid offers, quarantine of the corrupt sample, missing file, missing field values, zero seats
`test_filters.py`	4	Destination and dates, budget, direct-only, mandatory flexibility
`test_ranker_and_normalizers.py`	6	Both normalisation directions, equal values, 24-hour clock, registry, ranking and explanation
`test_booking.py`	6	Booking lifecycle, seat exhaustion, invalid reference, cancellation, capacity after restart
`test_preference_quiz.py`	3	Ratings to weights, equal ratings, use in a request
`test_profile_divergence.py`	1	Different profiles produce different winners on the same data
`test_interactive_helpers.py`	4	Specification example request, combining limits, hard/soft summary, missing-file error

7.2 The specification's example request

To check the model against the scenario in the specification, the example request was entered in the interactive mode: Madrid to Malaga between 12 and 14 July, a maximum price of €50, at most one

transfer, a changeable ticket required and a morning departure, with the example weights from the specification (price 40%, duration 25%, departure 15% and flexibility 20%). Of the 15 offers to Malaga, nine survived the filter. The three SolRuta offers were removed because they are not changeable, and the three Camino Express offers were removed because they cost €50.35, which is €0.35 above the limit. Table 8 shows the top three offers.

Table 8. Top three offers for the specification's example request (scores 0–100; Dur. = duration, Dep. = departure time, Flex. = flexibility).

#	Offer and departure	Price	Duration	Fare	Price score	Dur. score	Dep. score	Flex. score	Total
1	CostaLink OFF-CL-1052, 12 Jul 09:15	€36.04	7 h 07 min	Changeable	70.0	62.5	96.9	0.0	58.2
2	VerdeBus OFF-VB-1055, 12 Jul 10:00	€39.75	6 h 55 min	Fully flexible	0.0	100.0	87.5	100.0	58.1
3	VerdeBus OFF-VB-1057, 14 Jul 10:00	€39.75	6 h 55 min	Fully flexible	0.0	100.0	87.5	100.0	58.1

CostaLink won because it is €3.71 cheaper than VerdeBus and departs closer to the preferred time, while VerdeBus is faster and fully flexible. The two offers are almost exactly balanced: the difference in their total scores is 0.03 points, and the program's explanation warned that the result was a close call. In my view both offers are reasonable answers to the request, and the program presents them as such. The example also shows the edge of a hard constraint, since the fastest offer on the route was excluded for being 35 cents over budget. Section 7.5 shows that this exclusion affected the result in an unexpected way.

7.3 Comparison of four traveller profiles

The specification asks for at least three traveller profiles to show that the same offers can produce different recommendations. Four profiles were compared on the same route and travel window as the example request. Their weights and hard constraints are given in Table 9, and the results in Table 10. The complete rankings, criterion scores and explanations are stored in `outputs/01_demo_four_profiles.txt`.

Table 9. Traveller profiles used in the comparison.

Profile	Weights	Hard constraints	Preferred departure
Budget backpacker	Price 60%, duration 15%, departure 15%, flexibility 10%	Price ≤ €45, at most 1 transfer	Morning (09:00)
Business executive	Duration 40%, departure 30%, flexibility 20%, price 10%	Direct only, changeable ticket	08:30
Eco-conscious traveller	Emissions 50%, rating 20%, duration 15%, price 15%	None beyond destination, dates and seats	Not weighted
Flexible tourist	Flexibility 45%, rating 20%, departure 20%, price 15%	Luggage ≥ 15 kg	Afternoon (14:00)

Table 10. Winning offer for each profile, Madrid to Malaga, 12–14 July 2026.

Profile	Offers left	Winner	Price	Duration	Fare	Score	Runner-up (score)
Budget backpacker	12 of 15	SolRuta OFF-SR-1046, 12 Jul 07:45	€34.15	7 h 48 min, 1 transfer	Not flexible	72.7	SolRuta OFF-SR-1047 (70.3)
Business executive	12 of 15	Camino Express OFF-CE-1049, 12 Jul 08:30	€50.35	6 h 26 min, direct	Fully flexible	90.0	VerdeBus OFF-VB-1055 (72.0)
Eco-conscious traveller	15 of 15	VerdeBus OFF-VB-1055, 12 Jul 10:00	€39.75	6 h 55 min, direct	Fully flexible	89.5	VerdeBus OFF-VB-1056 (89.5, tie)
Flexible tourist	12 of 15	VerdeBus OFF-VB-1056, 13 Jul 15:00	€39.75	6 h 55 min, direct	Fully flexible	92.5	VerdeBus OFF-VB-1055 (85.0)

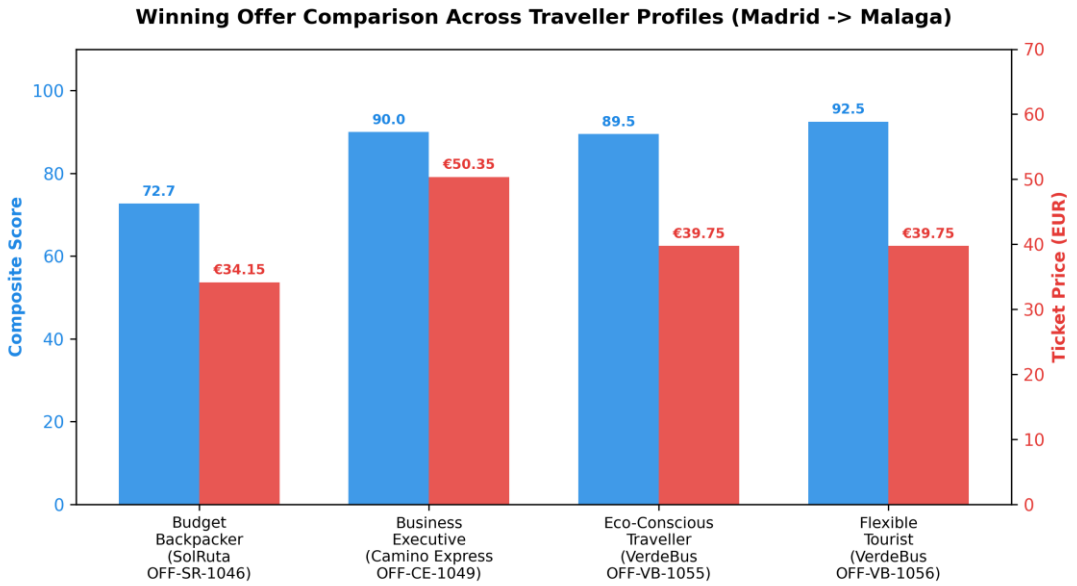


Figure 2. Total score and price of the winning offer for each profile. The scores are relative to each profile's own candidate pool and cannot be compared between profiles.

Three different companies won across the four profiles, and the two VerdeBus wins were different departures, so the model clearly responds to the traveller's preferences. The budget backpacker received the cheapest offer on the route. The business executive received the fastest direct offer, departing exactly at the preferred time, at the highest price; the explanation acknowledged that it costs €15.90 more than the cheapest alternative. The eco-conscious traveller received a VerdeBus offer with 7.4 kg of emissions per passenger, less than half of any other company on the route. The flexible tourist received the afternoon VerdeBus departure, which is fully flexible and closest to the preferred time.

7.4 Are the rankings reasonable?

I consider three of the four winners clearly reasonable. The business and flexible-tourist winners were ahead of their runners-up by 18.0 and 7.5 points, respectively, and both had the properties that the profiles describe. The eco-conscious result is reasonable as a choice of company, but the choice between the three VerdeBus departures is arbitrary: they are identical on every weighted criterion and every tie-break attribute, because this profile does not weight departure time. The program now says so explicitly. The evaluation revealed that an earlier version of the explainer described the 10:00 departure as "further from your preferred departure time", although departure time was not one of the profile's criteria. The explainer had treated the missing score as zero; this was corrected,

and the case is kept here as an example of why the explanation must be tested as carefully as the score.

The budget result is the most debatable. SolRuta won with a score of 72.7, but the third-placed CostaLink offer, at 69.9, is direct instead of requiring a transfer, 41 minutes faster, changeable instead of non-flexible and has more than twice the luggage allowance, for only €1.89 more. A real budget traveller might well prefer it. SolRuta won because of the price weight of 60%, and because min–max normalisation gave the cheapest offer 100 price points and CostaLink 66.2, turning a difference of €1.89 into a difference of 20.3 weighted points. This is consistent with the traveller's stated priorities, but it shows that the normalisation, and not only the weights, shapes the result.

7.5 Robustness checks

Because the example request was decided by a very small margin, three further checks were carried out. First, five percentage points of weight were moved between price and duration. With price at 35% and duration at 30%, VerdeBus won the example request by 5.3 points; with price at 45% and duration at 20%, CostaLink won by 5.4 points. A change in the weights that a traveller could hardly express precisely was enough to reverse the recommendation.

Second, the budget in the example request was raised from €50 to €51. This admits the three Camino Express offers, none of which wins. Nevertheless, the winner changed from CostaLink to VerdeBus, whose score rose from 58.1 to 72.9. The reason is that the most expensive offer in the pool defines the zero point of the price score. When Camino Express entered the pool at €50.35, VerdeBus at €39.75 was no longer the most expensive offer and its price score rose from 0 to 66.7. This phenomenon, in which adding or removing an alternative changes the order of the other alternatives, is known in decision analysis as rank reversal and was first discussed for the analytic hierarchy process by Belton and Gear (1983). It means that a traveller's recommendation can depend on offers they will never choose.

Third, all five profiles were run against all fifteen destinations, which gives 75 searches (Table 11). The business executive was always recommended Camino Express, and the flexible tourist and eco-conscious traveller were always recommended VerdeBus. The budget backpacker received SolRuta on 12 of 15 routes. NocheVia did not win a single search, and CostaLink won only one.

Table 11. Number of destinations (out of 15) on which each company produced the winning offer.

Profile	SolRuta	Camino Express	CostaLink	VerdeBus	NocheVia
Budget backpacker	12	1	1	1	0
Business executive	0	15	0	0	0
Eco-conscious traveller	0	0	0	15	0
Flexible tourist	0	0	0	15	0
Balanced (specification weights)	0	1	0	14	0
Total of 75 searches	12	17	1	45	0

8 Evaluation, limitations and possible unfair outcomes

The results answer the central question in the affirmative: a modular Python application can filter and rank bus offers according to individual requirements and preferences, and the recommendations change in sensible ways when the preferences change. The evaluation also showed several weaknesses that a traveller would not notice from the output alone. I discuss them below, ordered from the most to the least serious in my judgement.

8.1 Min–max normalisation amplifies small differences

Min–max normalisation always stretches the observed range to 0–100, regardless of how large the range is in reality. On the Malaga route, the prices that survived the budget filter varied by only €5.60, yet this was turned into the full range of 100 price points. A difference of €0.30 between SolRuta and NocheVia was worth 5.4 price points, and VerdeBus, which is the fastest, direct and fully flexible, finished last for the budget backpacker because it was the most expensive offer by €5.60. The choice of normalisation method is known to influence the ranking produced by multi-criteria methods (Jahan & Edwards, 2015). A fixed reference scale, such as scoring price relative to the traveller's budget rather than to the cheapest and most expensive offers, would keep small differences small.

8.2 Rankings depend on the candidate pool

Because every score is relative to the other offers in the pool, adding or removing an offer can change the order of the others, as the budget experiment in Section 7.5 demonstrated. This has two consequences. A traveller who widens the budget by one euro may receive a different recommendation for reasons that have nothing to do with the recommended offer, and a company could in principle influence the ranking of its competitors by adding or withdrawing an expensive offer. In addition, total scores cannot be compared across searches or profiles. The flexible tourist's winner scored 92.5 and the budget backpacker's 72.7, but this does not mean that the flexible tourist was better served; the scores were computed against different pools. Figure 2 therefore shows the scores side by side only with that caveat.

8.3 Hard constraints create sharp edges

A hard constraint treats an offer that misses the limit by one cent in the same way as one that misses it by fifty euros. In the example request, the fastest and most flexible offer on the route was excluded for being €0.35 above a budget of €50. This is the correct behaviour if the budget is a firm limit, and the program does show how many offers each constraint removed. However, it does not tell the traveller that a much better offer was only just outside the limit. A useful extension would be to report near misses, for example offers within 5% of a numeric limit, without including them in the ranking.

8.4 Partial credit for flexibility can be lost

The flexibility score was designed to give partial credit to fares that are changeable but not refundable. Normalisation removes this credit whenever the least flexible offer in the pool is changeable. In the business executive's search, every remaining offer was changeable because the profile requires it, so CostaLink and NocheVia received a flexibility score of 0, the same score that a completely rigid ticket would have received. Scoring flexibility on its fixed 0–1 scale, without min–max normalisation, would preserve the intended distinction.

8.5 Possible unfair outcomes

Friedman and Nissenbaum (1996) distinguish three kinds of bias in computer systems: preexisting bias, which has its roots in social institutions and practices; technical bias, which arises from technical constraints or considerations; and emergent bias, which arises in a context of use. Their main example is a travel booking system: an airline reservation system whose ranking algorithm systematically places some airlines' flights on the first screens and others' on later screens exhibits

technical bias, and they note that almost all bookings were made from the first screen. BusBidder Spain shows the same kind of technical bias. The clearest case concerns NocheVia, which did not win any of the 75 searches. Its departures at 22:30 and 23:45 are more than eight hours from every daytime preference, so it receives 0 departure points from any traveller who prefers the morning or afternoon. At the same time, the model has no criterion for the benefit that an overnight service offers, namely saving a night's accommodation and a day of travelling. The model therefore systematically disadvantages one type of service because of what it chooses to measure. CostaLink, whose main strength is its 25 kg luggage allowance, is disadvantaged in a similar way, since luggage is only a hard constraint and is never rewarded in the score.

If such a ranking were used in a real marketplace, the effect would not be limited to the companies. Travellers who do not state a departure preference still receive the default of "morning", so the default itself steers them away from overnight services. Customer ratings raise a related issue. In the simulated data each company has one fixed rating, so the rating criterion rewards the brand rather than the individual departure, and a new company without ratings could not be scored fairly at all. Finally, the single default value for missing emission data (Section 4.3) would make an unmeasured service look average, which may favour a polluting company that does not report its emissions over a cleaner one that does.

8.6 Limitations of the data and the implementation

The dataset is simulated and deterministic. Prices do not change between days or with demand, each company has one rating and one luggage allowance, and transfers add a surcharge but no extra travel time beyond the company's duration factor. The results therefore show how the model behaves, not how real Spanish bus companies compare. Within the implementation, the rejection statistics count only the first constraint that an offer breaks, so an offer that is both too expensive and not changeable is only counted once. The preference questionnaire has the weakness that a traveller who rates every factor as crucial obtains equal weights, which is an unopinionated ranking rather than the strong preference the traveller expressed; a pairwise method such as the analytic hierarchy process (Saaty, 1980) forces trade-offs, at the cost of more questions. Finally, the booking ledger is a single JSON file that is rewritten after each booking, which is adequate for one user but would not be safe if several sessions booked at the same time.

8.7 What I would change

Based on this evaluation, I would make four changes in a further version. I would score price and duration against fixed reference points, such as the traveller's budget and the fastest possible journey on the route, which would reduce both the amplification of small differences and the risk of rank reversal. I would score flexibility on its fixed scale. I would report offers that only just missed a numeric limit. And I would add a criterion for the value of overnight travel, or at least allow the traveller to say that they have no departure preference, so that overnight services are not excluded by default. The close-call warning that was added during the final review should also be kept, since it is the simplest way of telling the traveller that a recommendation is fragile.

9 Reproducibility

The application was developed and tested with Python 3.13 on Windows 11. It requires `rich` for the formatted tables (the program falls back to plain text tables if `rich` is missing) and `pytest` for the tests; `matplotlib` and `python-docx` are only needed to rebuild the figures and this report. All

dependencies are listed in `requirements.txt`. The dataset generator is deterministic and the ranking contains no random elements, so the results in Section 7 can be reproduced exactly. The only random element in the application is the booking reference. The commands below are run from the project folder and are also given in the README.

Table 12. Commands used to produce the evidence in this report.

Purpose	Command
Install dependencies	`python -m pip install -r requirements.txt`
Four-profile comparison (Section 7.3)	`python main.py --demo`
Interactive search and booking (Sections 6 and 7.2)	`python main.py --interactive`
Quarantine review (Section 4)	`python main.py --quarantine`
Automated tests (Section 7.1)	`python -m pytest -v`
Rebuild figures and report	`python report/generate_figures.py` and `python report/build_report.py`

10 Declaration of generative AI use

In accordance with Section 12 of the project specification, I declare the use of generative AI in this project. Two tools were used. Google Antigravity, an AI coding assistant, was used during development to suggest the module structure, to write a first version of the dataset generator and of several test cases, and to draft an earlier version of the report. Claude Code (Anthropic) was used at the end of the project to review the code and the report against the specification. This review found that the earlier report contained results that did not match the program, for example a winning SolRuta price of €29.15 that does not appear in the dataset. Claude Code was then used to correct these results, to add the persistence of seat capacity between sessions, the traveller's own hard constraints and the hard/soft summary in the interactive mode, the custom missing-file error, the comparison with the runner-up and the close-call warning in the explanation, and eight additional tests, and to help restructure and rewrite this report.

All generated code and text were checked before being kept. Every number in Sections 7 and 8 was produced by running the program and can be reproduced with the commands in Section 9, and the corresponding output files are included in the repository. The automated tests were run after each change, and selected scores were recalculated by hand to confirm the formulas, for example the price score of 70.0 for CostaLink in Table 8, which follows from (39.75 − 36.04) / (39.75 − 34.45) × 100. The references with a DOI were checked against the Crossref database.

Describe here, in your own words, which parts you designed, wrote or substantially changed yourself, which AI suggestions you rejected, and how you verified the final version. Section 12 of the specification requires this, and only you can state it accurately.

11 Conclusion

This project set out to answer how a Python application can filter and rank competing bus offers according to the requirements and preferences of an individual traveller. BusBidder Spain answers the question with a two-stage pipeline: hard constraints remove unacceptable offers, and a weighted sum of normalised criteria ranks the rest. The design separates loading, filtering, scoring, explanation and booking into independent, tested components, and uses descriptors and a decorator-based registry where they remove repetition and make the model extensible. The experiments

showed that the same 225 offers produce different and largely reasonable recommendations for different travellers.

The evaluation also showed that a transparent formula is not the same as a stable or a fair one. The specification's example request was decided by 0.03 points, a one-euro change in the budget changed the winner through rank reversal, and one company never won any of 75 searches because of what the model chooses to measure. The most important outcome of the project is therefore not the ranking itself, but the ability to explain and question it. A recommendation from BusBidder Spain is best understood as the best compromise under a traveller's stated constraints, weights and the chosen normalisation method, and the application now says so whenever the margin is small.

References

Belton, V., & Gear, T. (1983). On a short-coming of Saaty's method of analytic hierarchies. *Omega, 11*(3), 228–230. https://doi.org/10.1016/0305-0483(83)90047-6

Friedman, B., & Nissenbaum, H. (1996). Bias in computer systems. *ACM Transactions on Information Systems, 14*(3), 330–347. https://doi.org/10.1145/230538.230561

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Hwang, C.-L., & Yoon, K. (1981). *Multiple attribute decision making: Methods and applications*. Springer. https://doi.org/10.1007/978-3-642-48318-9

Jahan, A., & Edwards, K. L. (2015). A state-of-the-art survey on the influence of normalization techniques in ranking: Improving the materials selection process in engineering design. *Materials & Design, 65*, 335–342. https://doi.org/10.1016/j.matdes.2014.09.022

Meyer, B. (1988). *Object-oriented software construction*. Prentice Hall.

Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. *Artificial Intelligence, 267*, 1–38. https://doi.org/10.1016/j.artint.2018.07.007

Python Software Foundation. (n.d.). *Descriptor guide*. Python documentation. Retrieved October 30, 2026, from https://docs.python.org/3/howto/descriptor.html

Ramalho, L. (2022). *Fluent Python: Clear, concise, and effective programming* (2nd ed.). O'Reilly Media.

Saaty, T. L. (1980). *The analytic hierarchy process*. McGraw-Hill.

Triantaphyllou, E. (2000). *Multi-criteria decision making methods: A comparative study*. Springer. https://doi.org/10.1007/978-1-4757-3157-6

Appendix A. Evidence files

The following files in the `outputs/` folder were produced with the commands in Section 9 and contain the complete program output behind the tables and figures in this report.

Table 13. Evidence files in the outputs folder.

File	Content
`01_demo_four_profiles.txt`	Hard/soft summary, ranked offers, criterion scores and explanation for the four profiles (Section 7.3)
`02_specification_example_request.txt`	Interactive run of the specification's example request (Section 7.2)
`03_interactive_booking.txt`	Interactive search and booking of two seats, with the printed ticket (Section 6)
`04_capacity_after_restart.txt`	Seat capacity of the booked offer after the program is restarted (Section 6)
`05_quarantine_review.txt`	The five rejected rows of the corrupt sample and their reasons (Section 4)
`06_test_results.txt`	Verbose output of the 38 automated tests (Section 7.1)
`07_missing_file_error.txt`	Program behaviour when a data file is missing (Section 4.4)
`08_busbidder_log_excerpt.log`	Log entries written while the evidence above was produced